O‘ZBEKISTON RESPUBLIKASI RAQAMLI TEXNOLOGIYALAR VAZIRLIGI

MUHAMMAD AL-XORAZMIY NOMIDAGI

TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI



OPERATSION TIZIMLAR FANIDAN

Bajardi: Mamasoliyev Diyorbek Abdujalil o‘g‘li

Guruh: SRM002

Tekshirdi: Ro‘zibayev Ortik Baxtiyorovich

Toshkent – 2026

KIRISH

Zamonaviy operatsion tizimda bir vaqtning o'zida o'nlab jarayon protsessor vaqtiga da'vo qiladi. Protsessor esa har bir lahzada faqat bitta jarayonni bajara oladi. Shu sababli operatsion tizim tarkibida **rejalashtiruvchi** (*scheduler*) deb ataluvchi qism bo'lib, u navbatdagi jarayonlardan qaysi biri protsessorni olishini hal qiladi. Rejalashtirish algoritmining tanlovi tizimning javob berish tezligiga, protsessordan foydalanish darajasiga va jarayonlarning kutish vaqtiga bevosita ta'sir qiladi.

Ushbu amaliy ishda ikki xil rejalashtirish algoritmi — **ustuvorlik bo'yicha rejalashtirish** (*Priority Scheduling*, preemptive varianti) va **Round Robin** algoritmi — bir xil boshlang'ich ma'lumotlar asosida tadqiq qilinadi. Har bir algoritm avval qo'lda, qadamma-qadam hisoblanadi, so'ngra Python dasturlash tilida dasturi tuziladi va natijalar o'zaro solishtiriladi.

Ishning maqsadi: protsessorni rejalashtirish algoritmlarining ishlash tamoyilini o'rganish, berilgan jarayonlar to'plami uchun o'rtacha kutish vaqtini hisoblash va ikki algoritm samaradorligini taqqoslash.

Vazifalar:

1. rejalashtirishga oid asosiy tushunchalar va baholash mezonlarini o'rganish;
2. 23-variant jadvalini Priority Scheduling (preemptive) algoritmi bo'yicha qo'lda yechish;
3. xuddi shu jadvalni Round Robin algoritmi bo'yicha (TQ = 4) qo'lda yechish;
4. har ikki algoritm uchun Python dasturini tuzish va natijalarni qo'lda hisoblangan qiymatlar bilan tekshirish;
5. olingan natijalarni tahlil qilib, algoritmlarning afzallik va kamchiliklari bo'yicha xulosa chiqarish.

1. NAZARIY QISM

1.1. Jarayon va protsessorni rejalashtirish

Jarayon (*process*) — bajarilayotgan dastur nusxasi. Jarayon o'z hayoti davomida bir necha holatdan o'tadi: yangi, tayyor, bajarilmoqda, kutmoqda va

tugagan. Protsessor vaqtini kutayotgan jarayonlar **tayyor jarayonlar navbati**ga (*ready queue*) joylashtiriladi. Rejalashtiruvchi aynan shu navbatdan keyingi jarayonni tanlaydi.

Rejalashtirish algoritmlari ikki katta guruhga bo'linadi:

- **Preemptive** (*to'xtatuvchi*) algoritmlarda bajarilayotgan jarayon ishini tugatmasdan turib to'xtatilishi mumkin — masalan, ustuvorroq jarayon kelganda yoki ajratilgan vaqt tugaganda. To'xtatilgan jarayon tayyor navbatga qaytadi va keyinroq ishini davom ettiradi.
- **Non-preemptive** (*to'xtatmaydigan*) algoritmlarda protsessorni olgan jarayon to o'zi tugaguncha yoki kirish-chiqish amalini kutishga o'tguncha ishlayveradi.

1.2. Asosiy tushunchalar va baholash mezonlari

Algoritmlarni taqqoslash uchun har bir jarayon bo'yicha quyidagi vaqt ko'rsatkichlari hisoblanadi (1-jadval).

1-jadval

Rejalashtirish ko'rsatkichlari

Belgi	Inglizcha nomi	Ma'nosi
AT	Arrival Time	Jarayonning tizimga kelish vaqti
BT	Burst Time	Jarayonga kerak bo'lgan protsessor vaqti
CT	Completion Time	Jarayonning to'liq tugash vaqti
TAT	Turnaround Time	Jarayonning tizimda bo'lgan umumiy vaqti (aylanish vaqti)
WT	Waiting Time	Jarayonning navbatda kutib o'tkazgan vaqti
AWT	Average Waiting Time	Barcha jarayonlar kutish vaqtining o'rtachasi

Ko'rsatkichlar quyidagi formulalar bilan hisoblanadi:

$$TAT = CT - AT \tag{1}$$

$$WT = TAT - BT \tag{2}$$

$$AWT = (WT_1 + WT_2 + \ldots + WT_n) / n \tag{3}$$

bu yerda n — jarayonlar soni. (2) formulaning ma'nosi oddiy: jarayon tizimda bo'lgan vaqtidan o'zi ishlagan vaqtni ayirsak, qolgani — navbatda kutgan vaqt. **AWT qanchalik kichik bo'lsa, algoritm shunchalik samarali** hisoblanadi.

1.3. Gantt diagrammasi

Gantt diagrammasi — protsessor qaysi vaqt oralig'ida qaysi jarayonni bajarganini ko'rsatuvchi chiziqli diagramma. Har bir to'rtburchak bitta uzluksiz ishlash bo'lagini, uning ostidagi sonlar esa bo'lakning boshlanish va tugash vaqtini bildiradi. Har bir jarayonning tugash vaqti (CT) aynan shu diagrammadan olinadi.

1.4. Ustuvorlik bo'yicha rejalashtirish (Priority Scheduling)

Bu algoritmda har bir jarayonga **ustuvorlik soni** (*priority*) beriladi va protsessor tayyor jarayonlar ichidan ustuvorligi eng yuqori bo'lganiga ajratiladi. Odatda kichik son yuqori ustuvorlikni bildiradi. Ustuvorliklar teng bo'lsa, birinchi kelgan jarayon tanlanadi.

Preemptive variantda har safar yangi jarayon kelganda tanlov qayta tekshiriladi: yangi jarayonning ustuvorligi bajarilayotgan jarayonnikidan yuqori bo'lsa, joriy jarayon to'xtatiladi va protsessor yangi jarayonga beriladi.

Algoritmning asosiy kamchiligi — **ochlik** (*starvation*): past ustuvorlikka ega jarayon yuqori ustuvorli jarayonlar oqimi tufayli juda uzoq kutib qolishi mumkin. Bunga qarshi **qarish** (*aging*) usuli qo'llanadi — uzoq kutgan jarayonning ustuvorligi asta-sekin oshirib boriladi.

Adabiyotlarda ta'kidlanishicha, eng qisqa ishni birinchi bajarish (SJF) algoritmi ustuvorlik algoritmining xususiy holidir: unda ustuvorlik sifatida jarayonning protsessor vaqti olinadi [1]. Uning preemptive varianti — **SRTF** (*Shortest Remaining Time First*) — ustuvorlik sifatida jarayonning **qolgan** bajarilish vaqtidan foydalanadi. Bu bog'liqlik 3-bo'limda qo'llanilgan.

1.5. Round Robin algoritmi

Round Robin (*aylanma navbat*) algoritmi vaqtni taqsimlash tizimlari uchun ishlab chiqilgan. Har bir jarayonga navbat bilan bir xil vaqt bo'lagi — **vaqt kvanti** (*time quantum*, TQ) ajratiladi. Algoritm quyidagi qoidalar asosida ishlaydi:

1. tayyor jarayonlar FIFO (birinchi kelgan — birinchi chiqadi) tamoyilidagi navbatda turadi;
2. navbat boshidagi jarayon ko'pi bilan TQ vaqt birligi ishlaydi;

3. jarayon kvant ichida tugasa, protsessor darhol navbatdagi jarayonga o'tadi;
4. kvant tugab, jarayon tugamagan bo'lsa, u to'xtatiladi va navbat oxiriga qo'yiladi;
5. kvant davomida yangi jarayon kelsa, u to'xtatilgan jarayondan oldin navbatga qo'shiladi.

Kvant kattaligi muhim ahamiyatga ega: juda katta TQ da algoritm FCFS ga aylanib qoladi, juda kichik TQ da esa jarayonlarni almashtirish (*context switch*) soni keskin oshib, protsessor vaqtining katta qismi foydasiz sarflanadi [1, 2].

2. BOSHLANG'ICH MA'LUMOTLAR

Tartib raqami 23 bo'lgani uchun topshiriqlar ro'yxatidagi 23-jadval tanlandi (2-jadval). Ikkala topshiriq ham shu jadval asosida bajariladi.

2-jadval

23-variant jarayonlari

Jarayon	Kelish vaqti (AT)	Bajarilish vaqti (BT)
P1	0	14
P2	5	8
P3	4	2
P4	3	3
P5	1	9

Algoritm varianti. Topshiriq shartiga ko'ra toq tartib raqamli talabalar preemptive variantni bajaradi. 23 — toq son, shuning uchun 1-topshiriq **Priority Scheduling (preemptive)** algoritmi bo'yicha bajariladi.

Vaqt kvanti. Shartga ko'ra TQ tartib raqamni 3 ga bo'lgandagi qoldiqqa 2 ni qo'shish orqali topiladi:

$$TQ = 23 \bmod 3 + 2 = 2 + 2 = 4 \tag{4}$$

Ustuvorlik haqida qabul qilingan shart. Berilgan jadvalda ustuvorlik ustuni mavjud emas. Shu sababli 1.4-bo'limda keltirilgan yondashuvga tayanib, ustuvorlik sifatida jarayonning **qolgan bajarilish vaqti** olindi: qolgan vaqti qancha kam bo'lsa, ustuvorligi shuncha yuqori. Qolgan vaqtlar teng bo'lganda **avval kelgan** (AT kichik) jarayon tanlanadi.

3. 1-TOPSHIRIQ: PRIORITY SCHEDULING (PREEMPTIVE)

3.1. Qadamma-qadam yechim

Yechish jarayonida har bir yangi jarayon kelgan va har bir jarayon tugagan lahzada tayyor jarayonlar ro'yxati tuzilib, ulardan qolgan vaqti eng kichigi tanlanadi. 3-jadvalda qavs ichida jarayonning shu lahzadagi qolgan vaqti ko'rsatilgan.

3-jadval

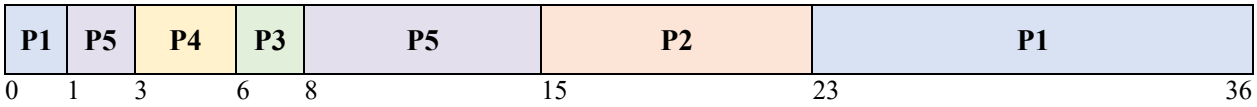
1-topshiriqning qadamma-qadam yechimi

Vaqt oralig'i	Tayyor jarayonlar (qolgan vaqt)	Tanlandi	Sabab
0–1	P1(14)	P1	Faqat P1 kelgan
1–3	P1(13), P5(9)	P5	9 < 13, P1 to'xtatildi
3–4	P1(13), P5(7), P4(3)	P4	3 eng kichik, P5 to'xtatildi
4–6	P1(13), P5(7), P4(2), P3(2); t = 5 da P2(8)	P4	P4 = P3 = 2, P4 oldin kelgan (AT 3 < 4)
6–8	P1(13), P5(7), P3(2), P2(8)	P3	2 eng kichik
8–15	P1(13), P5(7), P2(8)	P5	7 eng kichik
15–23	P1(13), P2(8)	P2	8 < 13
23–36	P1(13)	P1	Faqat P1 qoldi

Eng muhim qaror 4-lahzada qabul qilinadi: shu paytda P4 va yangi kelgan P3 ning qolgan vaqtlari teng (2 birlik). Qabul qilingan shartga ko'ra avval kelgan P4 ishini davom ettiradi. 5-lahzada kelgan P2 (8 birlik) P4 ning qolgan 1 birligidan katta bo'lgani uchun uni to'xtata olmaydi.

3.2. Gantt diagrammasi

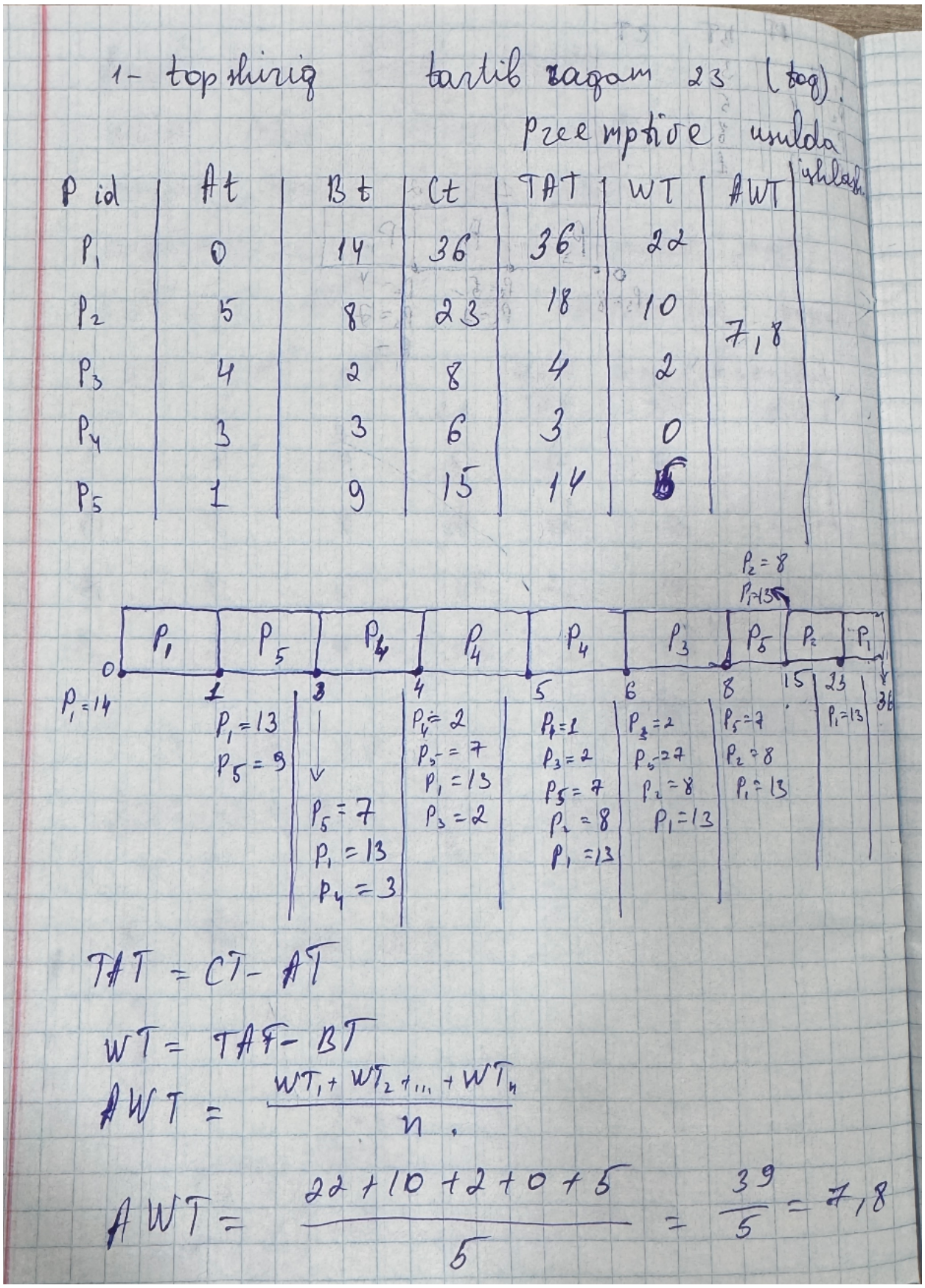
Qadamma-qadam yechim asosida qurilgan Gantt diagrammasi 1-rasmda keltirilgan.



1-rasm. 1-topshiriq bo'yicha Gantt diagrammasi

3.3. Qo'lda hisoblash

Daftarda bajarilgan qo'lda yechim 2-rasmda keltirilgan.



2-rasm. 1-topshiriqning qo'lda yechimi

Diagrammadan olingan tugash vaqtlari asosida (1) va (2) formulalar bo'yicha hisoblangan natijalar 4-jadvalda keltirilgan.

4-jadval

1-topshiriq natijalari

Jarayon	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	14	36	36 − 0 = 36	36 − 14 = 22
P2	5	8	23	23 − 5 = 18	18 − 8 = 10
P3	4	2	8	8 − 4 = 4	4 − 2 = 2
P4	3	3	6	6 − 3 = 3	3 − 3 = 0
P5	1	9	15	15 − 1 = 14	14 − 9 = 5
Jami				**75**	**39**

(3) formula bo'yicha:

$$AWT = (22 + 10 + 2 + 0 + 5) / 5 = 39 / 5 = 7{,}8 \quad (5)$$

$$ATAT = (36 + 18 + 4 + 3 + 14) / 5 = 75 / 5 = 15{,}0 \quad (6)$$

3.4. Dasturiy amalga oshirish

Dastur Python 3 tilida yozilgan (`preemptive_23.py` fayli). Algoritm vaqtni **bir birlikdan** oshirib boradi va har bir birlikda qaror qayta qabul qilinadi — bu preemptive rejalashtirishning dasturiy modeli. Dasturning asosiy qismi quyidagicha:

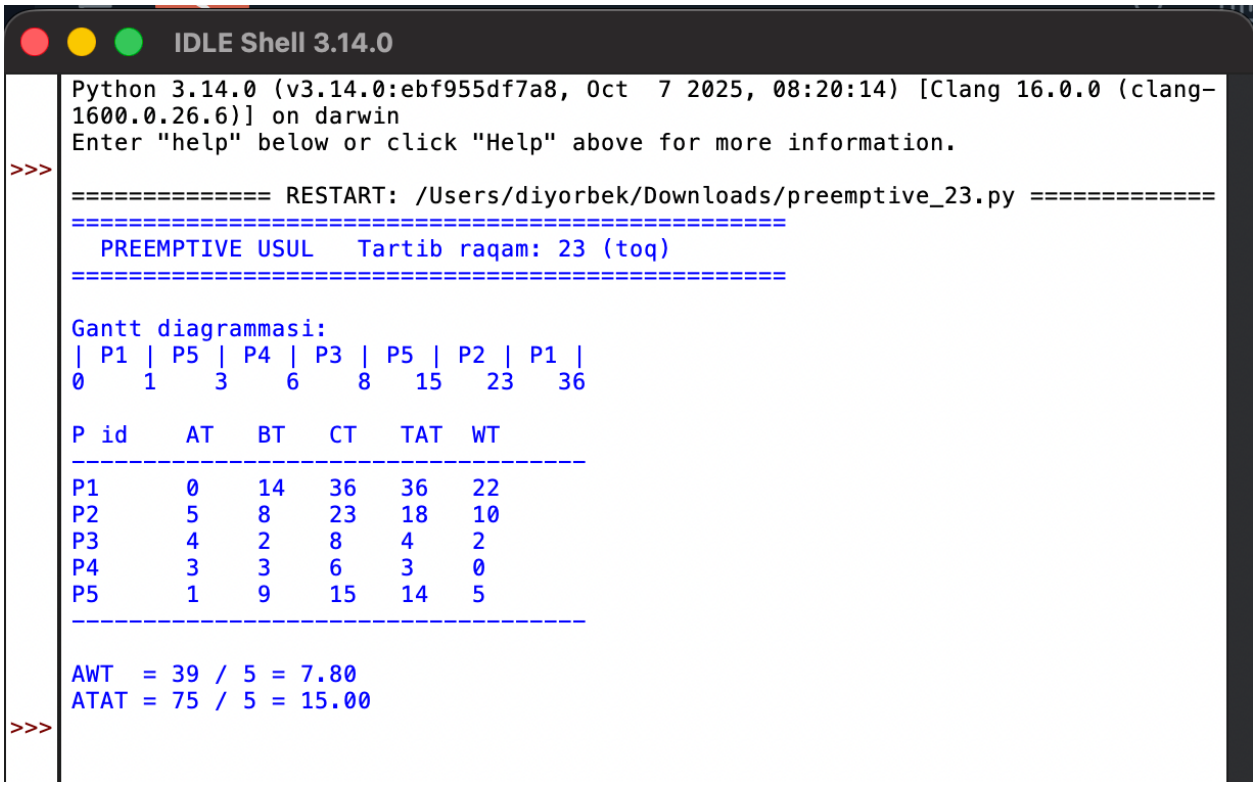
```
tayyor = [i for i in range(n) if AT[i] <= vaqt and qolgan[i] > 0]
i = min(tayyor, key=lambda j: (qolgan[j], AT[j]))
qolgan[i] -= 1
vaqt += 1
```

- birinchi qator shu lahzagacha kelgan va hali tugamagan jarayonlar ro'yxatini (tayyor navbatni) tuzadi;
- ikkinchi qator ulardan qolgan vaqti eng kichigini tanlaydi; `key` ichidagi ikkinchi element `AT[j]` tenglik holatida avval kelgan jarayonni tanlashni ta'minlaydi;
- keyingi ikki qator tanlangan jarayonni bir birlik bajaradi: uning qolgan vaqti kamayadi, tizim vaqti esa oshadi.

Jarayonning qolgan vaqti nolga yetganda tugash vaqti `CT` ga yoziladi. Barcha jarayonlar tugagach, dastur (1)–(3) formulalar bo'yicha TAT, WT va AWT ni hisoblab, Gantt diagrammasi va natijalar jadvalini chiqaradi.

3.5. Dastur natijasi

Dasturning terminaldagi natijasi 3-rasmda keltirilgan.



3-rasm. preemptive_23.py dasturining natijasi

Dastur chiqargan Gantt diagrammasi, tugash vaqtlari va AWT = 7,80 qiymati qo'lda hisoblangan natijalar bilan to'liq mos keladi.

4. 2-TOPSHIRIQ: ROUND ROBIN ALGORITMI (TQ = 4)

4.1. Qadamma-qadam yechim

Round Robin'da qarorni ustuvorlik emas, **navbatdagi o'rin** belgilaydi: kim navbatga oldin turgan bo'lsa, o'sha oldin ishlaydi. Shu sababli 5-jadvalda har bir oraliqdan keyingi navbat holati ko'rsatilgan.

5-jadval

2-topshiriqning qadamma-qadam yechimi

Vaqt	Ishlaydi	Ishladi	Qoldi	Shu oraliqda keldi	Navbat (oraliq oxirida)
0–4	P1	4	10	P5, P4, P3	P5, P4, P3, P1
4–8	P5	4	5	P2	P4, P3, P1, P2, P5
8–11	P4	3	0 (tugadi)	—	P3, P1, P2, P5

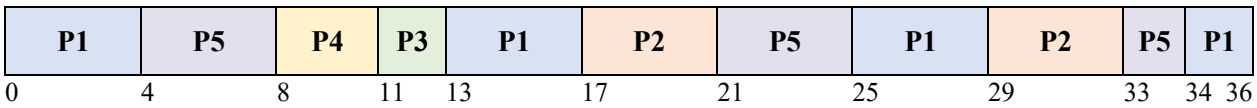
Vaqt	Ishlaydi	Ishladi	Qoldi	Shu oraliqda keldi	Navbat (oraliq oxirida)
11–13	P3	2	0 (tugadi)	—	P1, P2, P5
13–17	P1	4	6	—	P2, P5, P1
17–21	P2	4	4	—	P5, P1, P2
21–25	P5	4	1	—	P1, P2, P5
25–29	P1	4	2	—	P2, P5, P1
29–33	P2	4	0 (tugadi)	—	P5, P1
33–34	P5	1	0 (tugadi)	—	P1
34–36	P1	2	0 (tugadi)	—	—

Jadvaldan ikki muhim holatni ajratib ko'rsatish lozim:

- **4-lahza.** P1 birinchi kvantini tugatadi va navbat oxiriga o'tadi. Shu kvant davomida kelgan P5, P4 va P3 navbatga P1 dan oldin qo'shiladi. P3 aynan 4-lahzada kelgani uchun ham 5-qoidaga ko'ra P1 dan oldin turadi.
- **5-lahza.** P2 keladi va navbatda allaqachon turgan P1 dan keyin joylashadi. Shuning uchun 13-lahzada P2 emas, P1 ishlaydi: P1 navbatga 4-lahzada, P2 esa 5-lahzada turgan.

4.2. Gantt diagrammasi

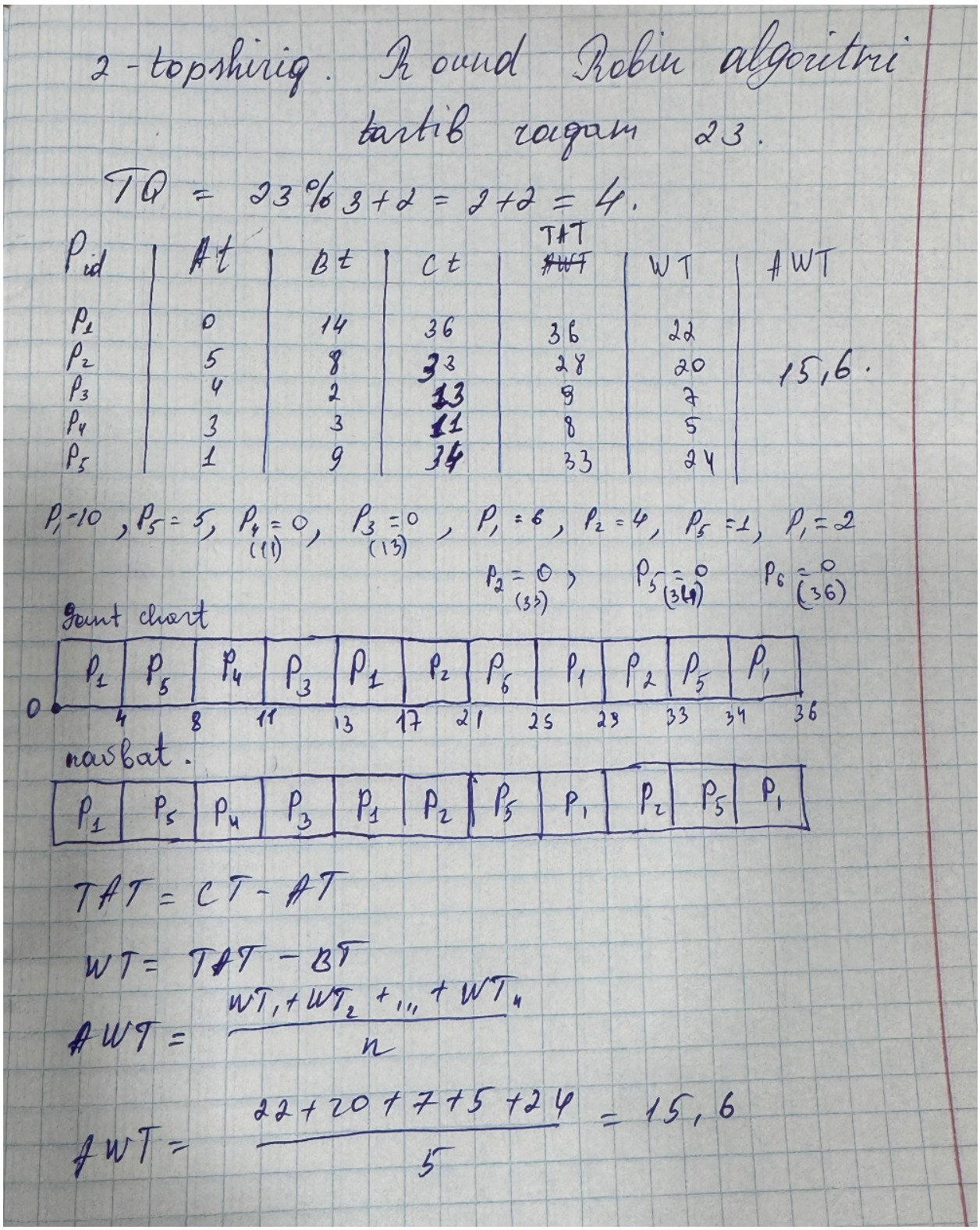
2-topshiriq bo'yicha Gantt diagrammasi 4-rasmda keltirilgan.



4-rasm. 2-topshiriq bo'yicha Gantt diagrammasi (TQ = 4)

4.3. Qo'lda hisoblash

Daftarda bajarilgan qo'lda yechim 5-rasmda keltirilgan.



5-rasm. 2-topshiriqning qo'lda yechimi

Hisoblash natijalari 6-jadvalda keltirilgan.

6-jadval

2-topshiriq natijalari

Jarayon	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	14	36	36 − 0 = 36	36 − 14 = 22

Jarayon	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P2	5	8	33	33 − 5 = 28	28 − 8 = 20
P3	4	2	13	13 − 4 = 9	9 − 2 = 7
P4	3	3	11	11 − 3 = 8	8 − 3 = 5
P5	1	9	34	34 − 1 = 33	33 − 9 = 24
Jami				**114**	**78**

$$AWT = (22 + 20 + 7 + 5 + 24) / 5 = 78 / 5 = 15,6 \quad (7)$$

$$ATAT = (36 + 28 + 9 + 8 + 33) / 5 = 114 / 5 = 22,8 \quad (8)$$

4.4. Dasturiy amalga oshirish

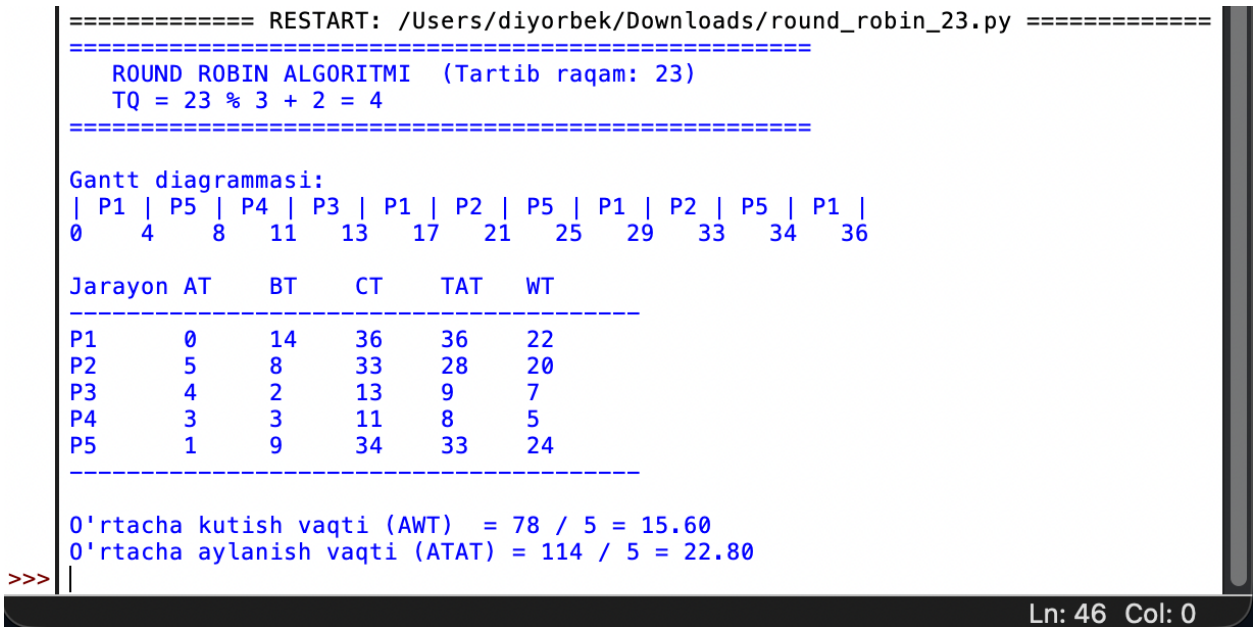
Dastur `round_robin_23.py` faylida joylashgan. Tayyor navbat Python'ning `collections.deque` tuzilmasi yordamida modellashtirilgan [4]: `popleft()` navbat boshidan jarayon oladi, `append()` esa navbat oxiriga qo'shadi. Har ikki amal O(1) vaqtda bajariladi. Vaqt kvanti dasturda formula bo'yicha hisoblanadi: `TQ = tartib_raqam % 3 + 2`. Bitta kvantni bajarish qismi:

```
i = navbat.popleft()
ishlash = min(TQ, qolgan[i])
vaqt += ishlash
qolgan[i] -= ishlash
kelganlarni_qosh(vaqt)
if qolgan[i] > 0:
    navbat.append(i)
```

- `min(TQ, qolgan[i])` — jarayon kvantdan kam vaqt talab qilsa, faqat kerakli vaqtcha ishlaydi (masalan, P4 — 3 birlik, P3 — 2 birlik);
- `kelganlarni_qosh(vaqt)` kvant davomida kelgan jarayonlarni navbatga qo'shadi va u ataylab to'xtatilgan jarayonni navbatga qaytarishdan **oldin** chaqiriladi — bu 1.5-bo'limdagi 5-qoidaning dasturiy ifodasi;
- jarayon tugamagan bo'lsa, `navbat.append(i)` uni navbat oxiriga qo'yadi, aks holda uning tugash vaqti saqlanadi.

4.5. Dastur natijasi

Dasturning terminaldagi natijasi 6-rasmda keltirilgan.



6-rasm. round_robin_23.py dasturining natijasi

Dastur natijasi (AWT = 15,60) qo'lda hisoblangan qiymat bilan mos keladi.

5. NATIJALAR TAHLILI

Ikkala algoritmning bir xil jadval bo'yicha olingan natijalari 7-jadvalda solishtirilgan.

7-jadval

Algoritmlarni taqqoslash

Ko'rsatkich	Priority (preemptive)	Round Robin (TQ = 4)
O'rtacha kutish vaqti (AWT)	7,8	15,6
O'rtacha aylanish vaqti (ATAT)	15,0	22,8
Eng uzoq kutgan jarayon	P1 — 22	P5 — 24
Jarayonlar almashinuvi soni	6	10
Barcha jarayonlar tugash vaqti	36	36

Tahlil natijasida quyidagilar aniqlandi:

- **Kutish vaqti.** Priority (preemptive) algoritmida o'rtacha kutish vaqti Round Robin'ga nisbatan ikki baravar kam (7,8 va 15,6). Sababi — qisqa jarayonlar (P3, P4) kelishi bilan bajarilib, uzun jarayonlarni kutib o'tirmaydi. P4 umuman kutmagan (WT = 0).
- **Adolatlilik.** Round Robin'da protsessor vaqti barcha jarayonlar o'rtasida teng bo'linadi. Priority algoritmida esa eng uzun jarayon P1 butun vaqt

davomida orqaga surilib, faqat 23-lahzada ishini davom ettirdi. Jarayonlar soni ko'paysa, bu holat ochlikka (starvation) olib kelishi mumkin.

- **Qo'shimcha xarajat.** Round Robin'da jarayonlar 10 marta, Priority algoritmida esa 6 marta almashdi. Real tizimda har bir almashinuv (context switch) qo'shimcha vaqt talab qiladi, shuning uchun bu ko'rsatkich ham muhim.
- **Umumiy vaqt.** Ikkala holatda ham barcha jarayonlar 36-lahzada tugaydi, chunki protsessor bir lahza ham bo'sh turmagan va jami bajarilish vaqti 14 + 8 + 2 + 3 + 9 = 36 ga teng.

XULOSA

Amaliy ish davomida protsessorni rejalashtirishning ikki algoritmi — Priority Scheduling (preemptive) va Round Robin — nazariy jihatdan o'rganildi, 23-variant jadvali bo'yicha qo'lda yechildi va Python tilida dasturiy amalga oshirildi. Dastur natijalari qo'lda hisoblangan qiymatlar bilan to'liq mos keldi, bu esa yechimning to'g'riligini tasdiqlaydi.

Olingan natijalarga ko'ra, Priority (preemptive) algoritmi o'rtacha kutish vaqti bo'yicha samaraliroq (AWT = 7,8), Round Robin esa jarayonlarga protsessor vaqtini adolatli taqsimlaydi, lekin o'rtacha kutish vaqti yuqoriroq (AWT = 15,6). Demak, umumiy kutish vaqtini kamaytirish muhim bo'lgan tizimlarda ustuvorlikka asoslangan algoritmlar, foydalanuvchilar bilan muloqot qiluvchi interaktiv tizimlarda esa Round Robin afzalroq hisoblanadi.

Shuningdek, ish davomida ustuvorlik qiymatlari berilmaganda ularni qanday aniqlash masalasi ko'rib chiqildi va qolgan bajarilish vaqtidan ustuvorlik sifatida foydalanish SRTF algoritmiga mos kelishi ko'rsatildi.

FOYDALANILGAN ADABIYOTLAR

1. Silberschatz A., Galvin P. B., Gagne G. Operating System Concepts. 10th ed. — Hoboken: Wiley, 2018. — 1278 p.
2. Tanenbaum A. S., Bos H. Modern Operating Systems. 4th ed. — Boston: Pearson, 2015. — 1136 p.

3. Priority Scheduling in Operating System. — GeeksforGeeks. — URL: https://www.geeksforgeeks.org/operating-systems/priority-scheduling-in-operating-system/
4. collections — Container datatypes (deque). — Python 3 Documentation. — URL: https://docs.python.org/3/library/collections.html